higher education & training

Department:
Higher Education and Training
REPUBLIC OF SOUTH AFRICA

MARKING GUIDELINE

NATIONAL CERTIFICATE (VOCATIONAL)

PRINCIPLES OF COMPUTER PROGRAMMING
(Second Paper)
NQF LEVEL 3

26 NOVEMBER 2019

This marking guideline consists of 13 pages.

Copyright reserved

QUESTION 1: FUNCTIONS AND SUBPROCEDURES

1.1
```
Option Explicit On
Option Strict On

Public Class frmCalc
...
```

1.1.1
```
Private Sub frmMedAidContribution_Load(…) Handles
                                    MyBase.Load

    Call PopIncomeCategories()✓
```
Invoke subprocedure: Do NOT penalise if *Call* keyword is omitted. (1)

1.1.2
```
    Dim intCounter1, intCounter2 As Integer✓

    intCounter1 = 0      } ✓
    intCounter2 = 0
```
Variable declaration (both)
Initialise variables (both)
Any other appropriate variable names (2)

1.1.3
```
    Do While✓ (intCounter1 < strArrSpouse.Length)✓

        cboSpouse.Items.Add(strArrSpouse(intCounter1))✓
        intCounter1 = intCounter1 + 1✓

    Loop ✓
```
Pre-test *Do While* Loop
Assign array elements to combo box
Increment loop counter (5)

1.1.4
```
    For intCounter2 = 0 To (strArrParents.Length - 1)✓
            cboParents.Items.Add(strArrParents(intCounter2))✓

    Next
```
***For* loop**
Assign array elements to combo box (2)

1.1.5
```
    cboMemberIncome.SelectedIndex = 0
    cboSpouse.SelectedIndex = 0          } ✓
    cboParents.SelectedIndex = 0
```
Assign value of zero to *SelectedIndex* property (all three) (1)

1.1.6
```
    lblAdultPrice.Visible = True
```
Change *Visible* property of label control (1)

1.2

```
Private Sub chkAdult_CheckedChanged✓ (…) Handles
                                chkAdult.CheckedChanged

  If chkAdult.Checked = True Then✓
      grpAdultDependent.Enabled = True
  Else
      grpAdultDependent.Enabled = False✓

      cboSpouse.SelectedIndex = 0
      cboParents.SelectedIndex = 0✓
  End If
End Sub
```

Correct event procedure (CheckChanged)
Correct conditional test (If Else/End If)
Enable/Disable group box control (both)
Assign value of zero to *SelectedIndex* property (both) (4)

1.3

```
Private Sub chkChild_CheckedChanged(…) Handles
                                chkChild.CheckedChanged
  If chkChild.Checked = True Then
      grpChildDependent.Enabled = True
      lblChildPrice.Visible = True

      txtNumberChildren.Focus()✓
  Else
      grpChildDependent.Enabled = False
      lblChildPrice.Visible = False
  End If
End Sub
```

Focus text box control. (1)

NOTE TO MARKERS: QUESTION 1.2 and QUESTION 1.3 are marked in conjunction, therefore, only 1 mark should be assigned to QUESTION 1.3 for focusing the textbox control.

1.4 Select Case cboMemberIncome.SelectedIndex

```
Case 0
   lblAdultPrice.Text = "..."
   decMainMemberPrice = 0
   decAdultPrice = 0
   decChildPrice = 0

Case 1
   lblAdultPrice.Text = "Does not qualify"
   decMainMemberPrice = 0
   decAdultPrice = 0
   decChildPrice = 0

Case 2
   lblAdultPrice.Text = "R965 for an Adult Member"
   decMainMemberPrice = 965D
   decAdultPrice = 965D
   decChildPrice = 361D

Case 3
   lblAdultPrice.Text = "R1 344 for an Adult Member"
   decMainMemberPrice = 1344D
   decAdultPrice = 1344D
   decChildPrice = 395D

Case 4
   lblAdultPrice.Text = "R1 903 for an Adult Member"✓                (1)
   decMainMemberPrice = 1903D✓
   decAdultPrice = 1903D✓   (Mark any one)
   decChildPrice = 561D✓                                             (3)
End Select
```

Assign message to text property of label (all cases 0-4)
Assign correct amounts to all three variables (all cases 0-4)

NOTE TO MARKERS: The above question specifically instructs candidates to use a *Select Case* statement, therefore, an *If End If* statement cannot be marked.

1.5

```
Private Function CalcNoOfAdults() As Integer✓
  Dim intAdults As Integer

  If cboSpouse.SelectedIndex = 0✓ Then
      intAdults = 0
  Else                                    ✓
      intAdults = 1
  End If

  If cboParents.SelectedIndex = 0✓ Then
      intAdults = intAdults + 0
  Else
      intAdults = intAdults + cboParents.SelectedIndex    ✓
  End If

  Return intAdults✓
End Function
```

(1) (2) (2) (1)

ALTERNATIVE:

```
Private Function CalcNoOfAdults() As Integer✓
  Dim intAdults As Integer
  intAdults=0

  If cboSpouse.SelectedIndex <> 0✓ Then
       intAdults = 1 ✓
  End If

  If cboParents.SelectedIndex <> 0✓ Then
      intAdults = intAdults + cboParents.SelectedIndex  ✓
  End If

  Return intAdults✓
End Function
```

Correct function definition
Correct conditional test for *Spouse* combo box
Correct conditional test for *Parents* combo box
Return statement

1.6

```
Private Function CalcBasicMonthly(ByVal MainMemberPrice As Decimal, ByVal
NumberOfAdults As Integer, ByVal AdultPrice As Decimal, ByVal NumberofChild As
Integer, ByVal ChildPrice As Decimal) As Decimal✓✓

Return CDec((MainMemberPrice + (NumberOfAdults * AdultPrice) +
          (NumberofChild * ChildPrice)))✓✓
End Function
```

(2) (2)

Correct function definition with parameters (all five)
Return calculation

1.7
```
Private Function CalcDiscount(ByRef NumberofChild As Integer, ByVal
BasicMonthlyCost As Decimal) As Decimal✓✓                          (2)
    If NumberofChild >= 3 Then✓ ( OR  If NumberofChild > 2 Then✓)
       Return (BasicMonthlyCost * decDiscountPercentage)✓          (2)
    End If
End Function
```

Correct function definition with parameters (both)
Conditional test (If/End If)
Return calculation

1.8
```
Private Sub btnCalc_Click(…) Handles btnCalc.Click

 'Validate
 'Included validation
 If cboMemberIncome.SelectedIndex = 0 Then
                    cboMemberIncome.DroppedDown = True

 If chkAdult.Checked = True And cboSpouse.SelectedIndex = 0
                 And cboParents.SelectedIndex = 0 Then
    MessageBox.Show("No adult dependents selected")
    chkAdult.Checked = False
 End If

 If chkChild.Checked = True And CInt(txtNumberChildren.Text)
                                   < 1I Then
    MessageBox.Show("No child dependents selected")
    chkChild.Checked = False
 End If
```

1.8.1
```
If Integer.TryParse✓(txtNumberChildren.Text,0) = False Then✓
    MessageBox.Show("Number cannot be a decimal", "Input
                                          error")
    txtNumberChildren.SelectAll()                          ✓
    txtNumberChildren.Focus()
    Exit Sub
End If
```

Correct *TryParse* method
Communication and navigation (3)

1.8.2
```
intNoOfAdults = CalcNoOfAdults()✓
```

Invoke function and assign to variable (1)

1.8.3
```
intNoOfChildren = CInt(txtNumberChildren.Text)✓
```

Correct variable assignment (1)

1.8.4
```
decBasicMonthlyCost = CalcBasicMonthly(decMainMemberPrice,
                    intNoOfAdults, decAdultPrice,
                    intNoOfChildren, decChildPrice)
decDiscountAmount = CalcDiscount(intNoOfChildren,
                          decBasicMonthlyCost)
```

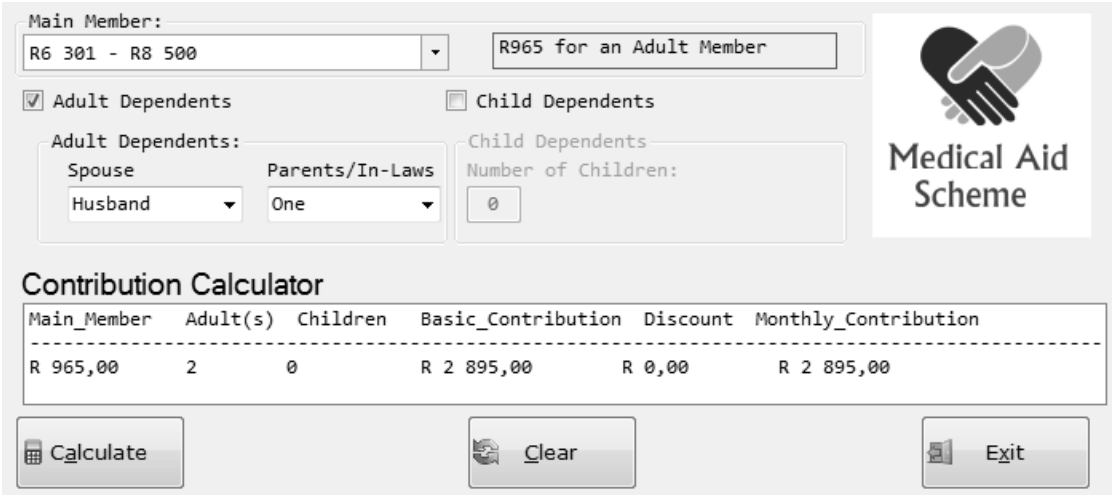
Invoke functions (both) (3)

Correct input parameters (both functions)
Assignment to variables

1.8.5
```
decFinalMonthlyCost = decBasicMonthlyCost –
                                    decDiscountAmount ✓
```
Correct calculation and assignment
```
  'Output to list box
   DisplayHeading()
   DrawLine()
   DisplayDataLine()
End Sub
```
(1)

1.9 Printout of source code

1.10



(1)
[45]

QUESTION 2: ARRAYS

2.1
```
Private Sub Initialisation()
    Dim i As Integer = 0
```

2.1.1
```
    Do While (i < sngArrTotalPerYear.Length)✓
```
(1)

2.1.2
```
        sngArrTotalPerYear(i) = 0.0✓
        i = i + 1
    Loop
    strWettestMonth = ""
    sngHighestFigure = 0.0                    ✓
End Sub
```
(2)

***Do While* loop**
Assign 0.0 to array
Initialise variables (both)

2.2 Private Sub CheckForWettestMonth(ByVal value As Single, ByVal index As Integer)✓

2.2.1
```
Dim strArrMonthNames() As String = {"Jan", "Feb",
                                    "Mar", "Apr",
                                    "May", "Jun"}✓
```
(2)

2.2.2
```
If value > sngHighestFigure Then✓
```

(a)
```
    sngHighestFigure = value✓
```

(b)
```
strWettestMonth = strArrMonthNames(index)✓
    End If
End Sub
```
(3)

Subprocedure declaration (correct parameters)
Array declaration
Conditional test (If/End If)
Correct assignments

2.3 Private Function CalcYearlyTotalRainfall(ByVal myArray()✓ As Single) As Single (1)

```
Dim j As Integer
Dim total As Single = 0.0
```

2.3.1
```
For j = 0 To myArray.Length – 1✓
   total = total + myArray(j)✓
   Call CheckForWettestMonth(myArray(j)✓, j)✓
Next
```
(4)

2.3.2
```
Return total✓
End Function
```
(1)

Array as parameter
***For Next* loop**
Correct assignment
Procedure call
Return statement

2.4
```
sngArrTotalPerYear(0)✓ =
        CalcYearlyTotalRainfall ✓ (sngArr2014Figures)✓
sngArrTotalPerYear(1) =
        CalcYearlyTotalRainfall(sngArr2015Figures)
sngArrTotalPerYear(2) =
        CalcYearlyTotalRainfall(sngArr2016Figures)
sngArrTotalPerYear(3) =
        CalcYearlyTotalRainfall(sngArr2017Figures)
sngArrTotalPerYear(4) =
        CalcYearlyTotalRainfall(sngArr2018Figures)
```
(3)

Assign to array (sngTotalPerYear)
Invoke procedure (any one)
Correct parameter for procedure

Copyright reserved

2.5
```
Private Function DetermineDriestYear() As String
  Dim i As Integer = 0
  Dim position As Integer
  Dim strYears() As String = {"2014", "2015", "2016", "2017",
                              "2018"}✓                                  (1)
  Dim lowestrainfall As Single = sngArrTotalPerYear(i)
    Do While (i < sngArrTotalPerYear.Length)✓
      If lowestrainfall > sngArrTotalPerYear(i) Then✓
          lowestrainfall = sngArrTotalPerYear(i)✓
          position = I✓                                                 (4)
      End If
       i = i + 1
    Loop
   Return strYears(position)✓
End Function                                                            (1)
```

Array declaration
Sequential search
Return statement

2.6
```
Private Function CalcMonthlyAvgRainfail() As Single
      Dim i As Integer = 0
      Dim total As Single = 0
      Dim average As Single = 0.0
      Dim numOfmonths As Integer = (intNoOfYears *
                                    intNoOfMonths)✓                     (1)
      Do While (i < sngArrTotalPerYear.Length)✓
         total = total + sngArrTotalPerYear(i✓)✓                        (3)
         i = i + 1
      Loop
      Try✓
         average = (total / numOfmonths)✓                               (2)
      Catch ex As Exception
         MessageBox.Show(ex.Message)
      End Try                                                           (1)
      Return average✓
End Function
```

Calculate number of months
Calculate rainfall totals (any loop structure acceptable)
Try Catch
Calculate average
Return statement

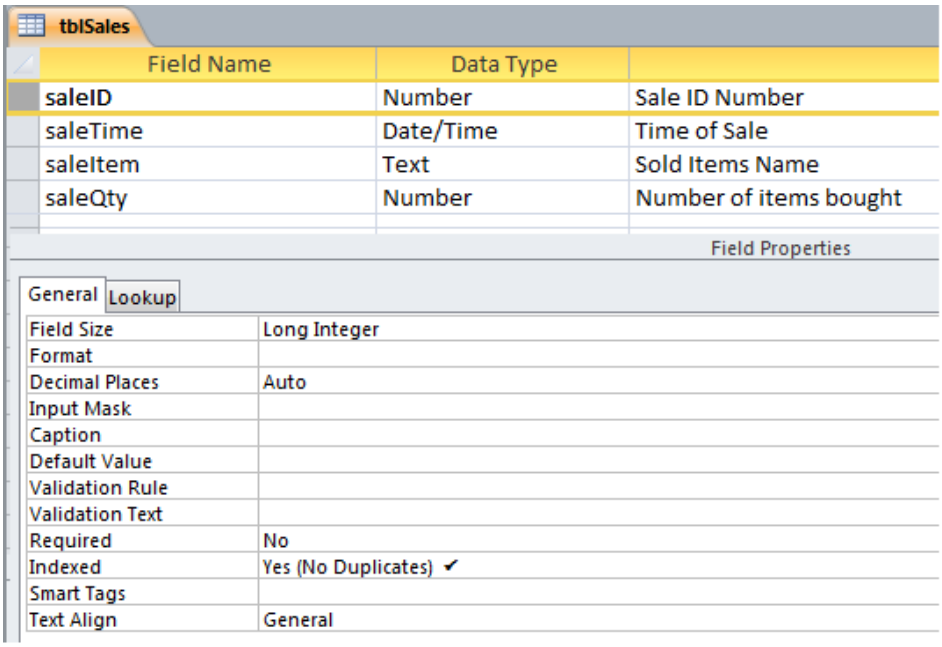
NOTE TO MARKERS: Assess candidate's logic for this question.

2.7 Printout of source code

[30]

QUESTION 3

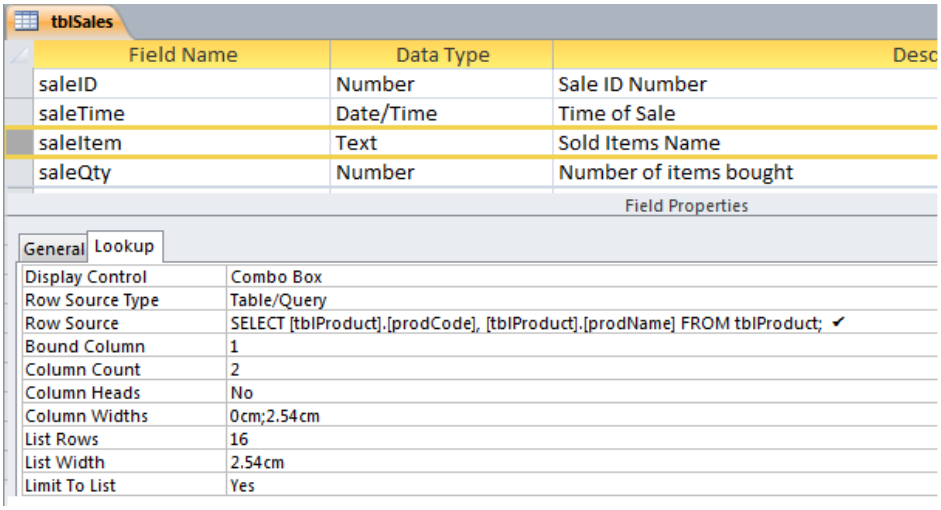
3.1 3.1.1



Correctly indexed

NOTE TO MARKERS: Do NOT allocate the above mark if the *saleID* field was made a primary key. (1)

3.1.2



Correct lookup (1)

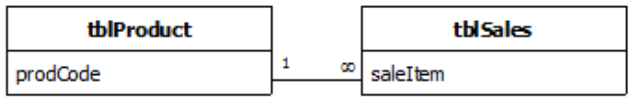
3.2

Columns

Name	Type	Size
prodCode	Text	4
prodName ✔	Text ✔	20 ✔
prodPrice	Double	8
prodQtyInStock	Integer	2

Relationships

tblProducttblSales



Attributes: Enforced ✔
RelationshipType: One-To-Many ✔

PRODUCTS

Correct field names
Correct data type
Correct size
Relationship type (1:M)
Enforced referential integrity

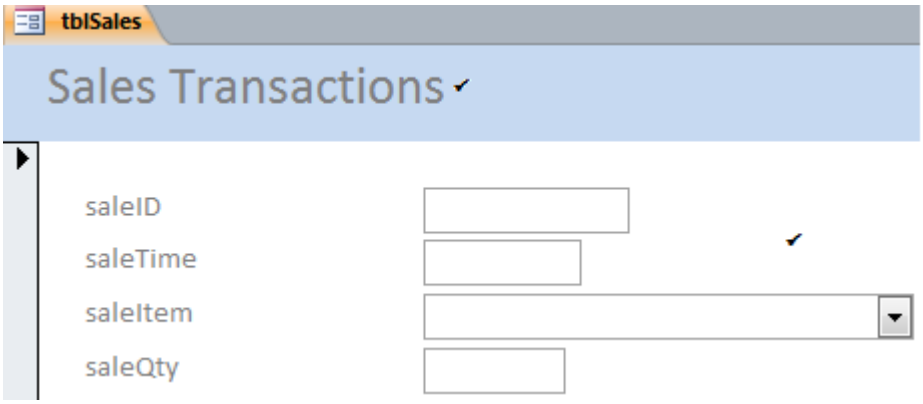
Columns

Name	Type	Size
saleID	Long Integer	4
saleTime ✔	Date/Time ✔	8 ✔
saleItem	Text	4
saleQty	Integer	2

SALES

Correct field names
Correct data type
Correct size (8)

3.3 3.3.3



Form design
Title (2)

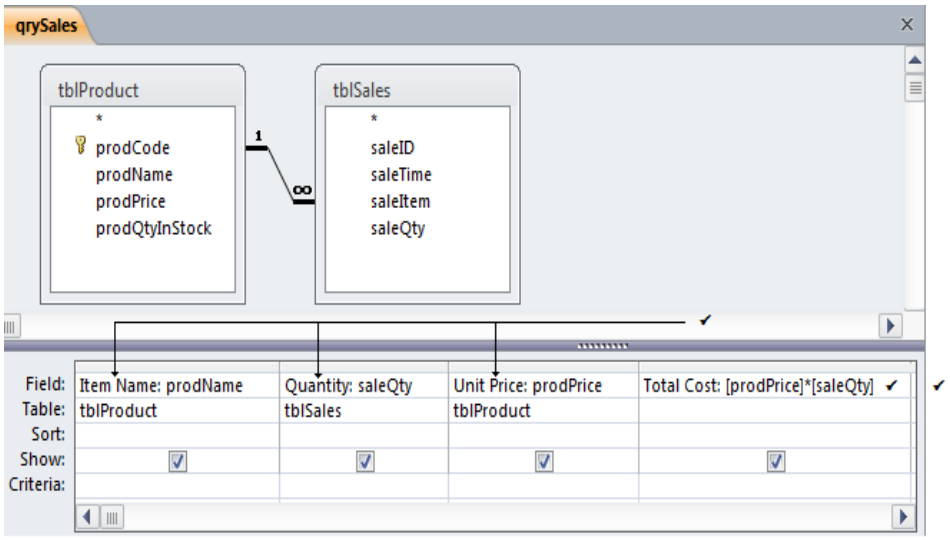
3.4 3.4.5

qrySales

Item Name	Quantity	Unit Price	Total Cost
Coke 330ml	2	11	22
Coke 330ml	3	11	33
Coke 330ml	2	11	22
Fanta 330ml	1	11	11
Fanta 330ml	1	11	11
Coke 2litre	2	19	38
Coke 2litre	2	19	38

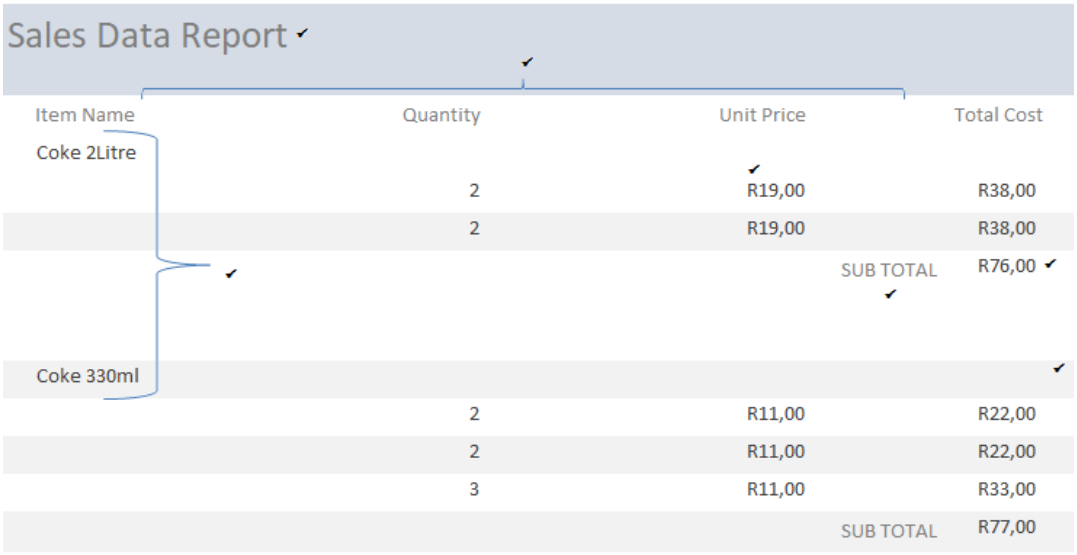
All fields
Calculated field {total cost}
Print records (3)

3.4.6



Changed field names
Calculated field formula
Print design view (3)

3.5



Sales Data Report

Item Name	Quantity	Unit Price		Total Cost
Coke 2Litre				
	2	R19,00		R38,00
	2	R19,00		R38,00
			SUB TOTAL	R76,00
Coke 330ml				
	2	R11,00		R22,00
	2	R11,00		R22,00
	3	R11,00		R33,00
			SUB TOTAL	R77,00

Correct fields
Grouping
Layout
Report title
Label (SUBTOTAL)
Currency symbol ("R")
Calculated subtotals (7)
[25]

TOTAL: 100

Copyright reserved